

4. Sockets

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

4.1 Vorschau

- Grundlagen
- Sockets
- Skalierbare Server-Architektur
 - Load-Balancer
 - Multi-Threading
 - Asynchrone Sockets
- Bibliotheken
- Zusammenfassung

4.1 Grundlagen

4.1 Grundlagen

- Cloud-Umgebung:
 - hohe Zuverlässigkeit: kaum Paketverluste, selten Netzwerk-Partitionierungen
 - vergleichsweise sichere Umgebung
- P2P/Internet:
 - geringere Zuverlässigkeit
 - nicht vertrauenswürdig
- Netzwerkcharakteristika

Netzwerk	Bandbreite (max.)	Latenz (min.)
LAN	400 GBit/s	1 μ s
Internet	1 GBit/s	1-100ms
WLAN	6,9 GBit/s	1ms
LTE (advanced)	1 GBit/s (downlink)	15ms

Protokolle aus Sicht von Java

Java APIs

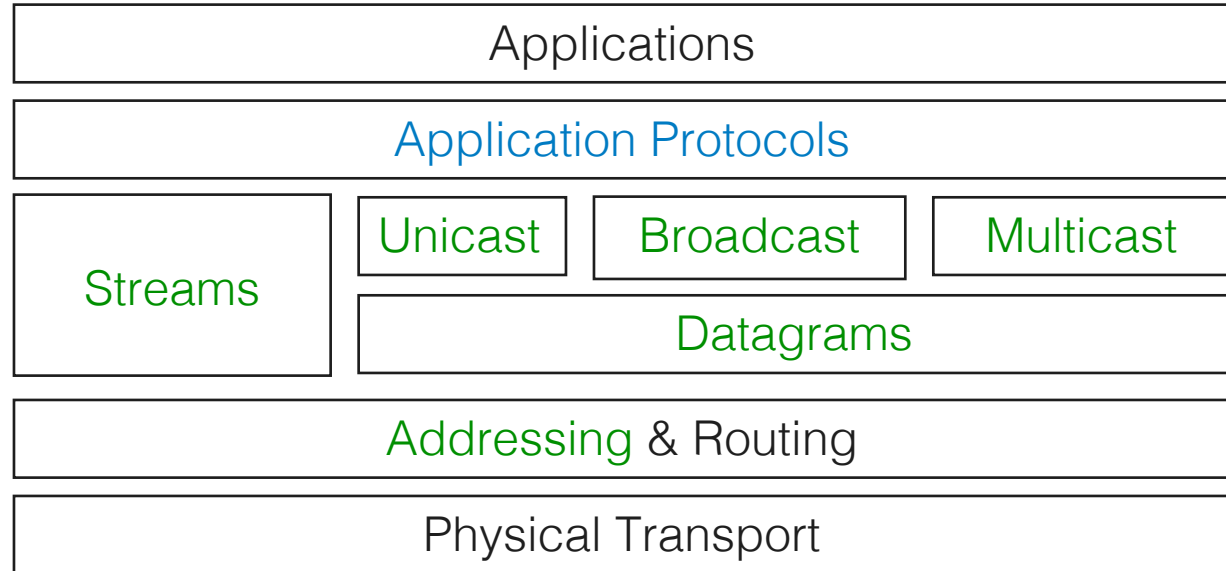
custom

1:1 or 1:Many

TCP, UDP

IP

Ethernet



Transfer Control Protocol (TCP)

- Streams: Nachrichten werden als Bytestrom versendet
 - Adressierung erfolgt durch eine Verbindung zwischen zwei Endpunkten
 - Erhält Sequenz der Nachrichten
 - Verlorene Nachrichten werden automatisch neu übertragen
- Weitere Aspekte (siehe Rechnernetze):
 - Flusskontrolle (engl. flow control): vermeidet Überlastung des Empfängers
 - Verstopfungskontrolle (engl. congestion avoidance): vermeidet Verstopfung im Netzwerk → Router kann überlastet sein, falls viele Verbindung über ihn gehen
- Beispiele: https, ssh, sftp, X11, ...



User Datagram Protocol (UDP)

- Datagrams: Nachrichten werden als Datagramme verschickt
- Verbindungslos
- Nachrichtensequenz nicht garantiert
- Unter Umständen Nachrichtenverluste
- Keine Fluss- und Verstopfungskontrolle
- Beispiele: Video- und Audioströme, tftp, ...

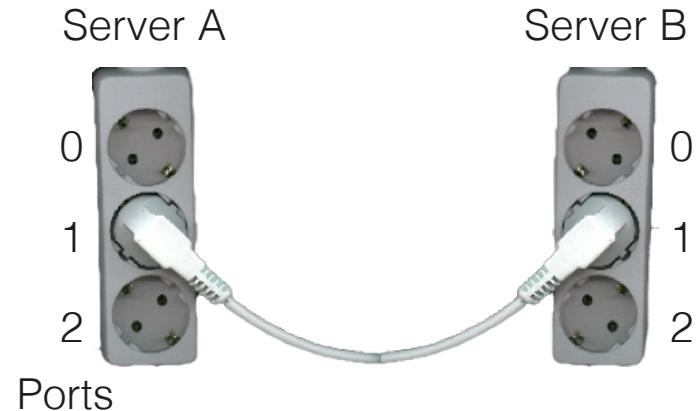


Internet Protocol (IP)

- Aufgaben: Adressierung und Routing
 - abstrahiert vom unterliegenden physikalischen Netz
 - Weglenkung (engl. routing)
- Adressen, um einen Rechner eindeutig zu identifizieren:
 - IPv4 verwendet 32 Bit Adressen, z.B. 192.168.70.30
 - IPv6 löst Knappheit der Adressen durch 128 Bit Adressierung
 - Meist werden Domain Names verwendet
 - Domain Name System (DNS) übernimmt Abbildung auf IP-Adressen

Internet Protocol (IP)

- Port-Nummern zur Identifikation eines bestimmten Diensts auf einem Rechner
 - 16 Bit Nummer
 - Prozess/Dienst muss sich beim System auf einem Port registrieren
 - Es gibt reservierte und frei verfügbare Ports
- Beispiele reservierter Ports:
 - Ports 22 SSH
 - Port 25 SMTP
 - Port 80 HTTP
 - Port 143 IMAP
 - ...



4.2 Sockets

4.2 Sockets

- Sockets sind Kommunikationsendpunkte
- Für eine nachrichtenbasierte Kommunikation zwischen Prozessen/Rechnern
- Für Java nur Internet-Protokollarchitektur
- (in C für verschiedene Protokoll-Stacks)

IP-Adressierung

- Java hat hierfür die Klasse `java.net.InetAddress`
- Hat drei Aufgaben:
 - Kapselt eine IP-Adresse
 - Kann einen DNS-Lookup durchführen (DomainName → IP-Adresse)
 - Bietet auch einen ReverseDNS-Lookup (IP-Adresse → DomainName)
- Auszug Methoden zum Erzeugen von `InetAddress`-Instanzen:
 - mit gegebener IP-Adresse: `InetAddress getByAddress(byte[] addr)`
 - mit DNS-Namen: `InetAddress getByName(String hostName)`
 - Localhost: `InetAddress getLocalHost()`

UDP - Beispiel

- Client und Server verwenden zum Senden/Empfangen die Klasse `DatagramSocket`
- Pakete sind Instanzen der Klasse `DatagramPacket`
- IP-Adresse und Port-Nummer in jedem Paket (da keine Verbindung vorhanden)
- Daten in Byte-Array

Beispiel: Server (Java UDP)

```
import java.io.*;    // IO exceptions
import java.net.*;   // sockets

public class UdpServer {

    public static void main(String[] args) {
        DatagramSocket socket; // UDP socket
        DatagramPacket packet; // UDP datagram
        String str;
        byte data[] = new byte[256];

        try {
            socket = new DatagramSocket(4711);
            while (true) {
                packet = new DatagramPacket(data, data.length);

                // blocking receive
                socket.receive(packet);
                str = new String(packet.getData(), 0, packet.getLength());

                System.out.println("received: "+str);
            }
            socket.close();
        } catch (IOException e) {}
    }
}
```

UdpServer.java

Beispiel: Client (Java UDP)

```
import java.net.* ; // sockets
import java.io.*;   // exceptions

public class UdpClient {

    public static void main (String[] args){
        byte[] data = "VerteilteSysteme".getBytes();
        String url  = "localhost";

        try {
            DatagramSocket socket = new DatagramSocket();
            DatagramPacket packet = new DatagramPacket( data, data.length, InetAddress.getByName(url), 4711);

            socket.send( packet); // blocking send (until data is copied into send buffer of socket)

            socket.close();
        }
        catch (IOException e) {
        }
    }
}
```

UdpClient.java

TCP in Java

- Client:
 - Verwendet die Klasse `Socket`
 - Konstruktor erhält IP-Adresse und Port-Nummer des Servers
 - Verbindungsaufbau erfolgt direkt beim Anlegen der `Socket`-Instanz
- Server:
 - Verwendet die Klasse `ServerSocket`, um Verbindungsanfragen anzunehmen
 - `accept()` blockiert bis eine Verbindungsanfrage eingeht und gibt eine `Socket`-Instanz zurück, für die anschließende Kommunikation mit einem Client
 - Rückgabetyp ist Instanz von der Klasse `Socket`

Beispiel: Simple EchoServer (Java TCP)

```
import java.net.*; import java.io.*;

public class TcpEchoServer {
    public static void main(String[] s){
        try {
            ServerSocket sock = new ServerSocket( 4713 ); // port = 4713
            Socket conn = sock.accept(); // block until connection request
            OutputStream out = conn.getOutputStream();
            InputStreamReader inp = conn.getInputStream();

            out.println( "Echo server active, enter 'bye' to exit.\r");
            do {
                idx = 0;
                do { // read one line, byte by byte
                    buffer[idx] = (byte) inp.read(); // read one byte
                    out.write( buffer[idx] );
                    idx++;
                } while(buffer[idx-1] != '\r');

                String msg = new String(buffer, 0, idx-1); // convert raw bytes to String
                System.out.println("received: " + msg);
                if ( msg.indexOf("bye") > 0 )
                    break;
            } while( true );
            conn.close( );
        } catch ( IOException ioX ) { ...
    }
}
```

TcpServer.java

Byte Order

- Die Reihenfolge in welcher die Bytes von Datentypen gespeichert werden, welche aus mehr als ein Byte bestehen, ist abhängig vom Prozessor.
 - Auch als **Host Byte Order** bezeichnet
 - x86-basierte Prozessoren verwenden Little Endian Byte Ordering
- TCP/IP verwendet hingegen Big Endian Byte Ordering
 - Auch als **Network Byte Order** bezeichnet
- Beispiel: `0x0000CAFE` (32-Bit, `uint_32`), 4 Bit für jede Hex-Ziffer

Big Endian Byte Ordering

Offset	0	1	2	3
Content	CA	FE	00	00

Little Endian Byte Ordering

Offset	0	1	2	3
Content	00	00	FE	CA

Byte Order (2)

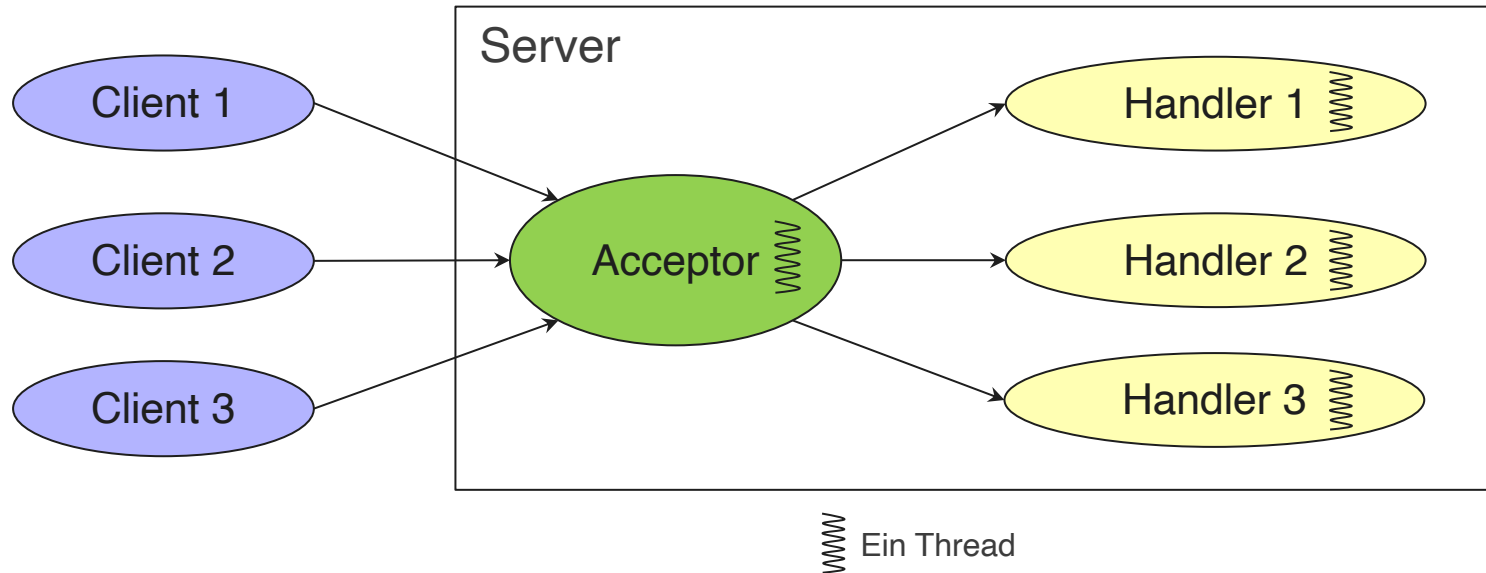
- Java-Streams verwenden Big Endian Byte Ordering
→ entspricht Network Byte Order
- Falls der Client und Server in Java geschrieben ist, muss nichts bzgl. der Byte-Order beachtet werden
- Falls Sockets in C benutzt werden, muss die Byte Order beachtet werden!
 - Die C-Socket-Bibliothek stellt Funktionen zur Konvertierung zwischen Host- und Network Byte Order bereit:

```
long  htonl (long value);      /* host to network long */
short htons (short value);     /* host to network short */
long  ntohl (long value);      /* network to host long */
short ntos  (short value);     /* network to host short */
```

- Achtung: auch bei der IP-Adresse und Port-Nummer beachten

Multi-Threaded Server für mehrere Clients

- Architektur: ein Thread pro Handler + ein Thread für den Acceptor



Multi-Threaded EchoServer (abstrakt)

- 1x Acceptor-Thread, Klasse EchoServer:

```
ServerSocket sock = new ServerSocket( 4713 );

do {
    Socket conn = sock.accept();           // block until connection request
    Handler handler = new Handler( conn );
    handler.start();                       // start handler thread
} while( true );
```

- 1 Handler-Thread pro Client, Klasse Handler:

```
public void run() {
    OutputStream out = conn.getOutputStream();
    InputStreamReader inp = conn.getInputStream();

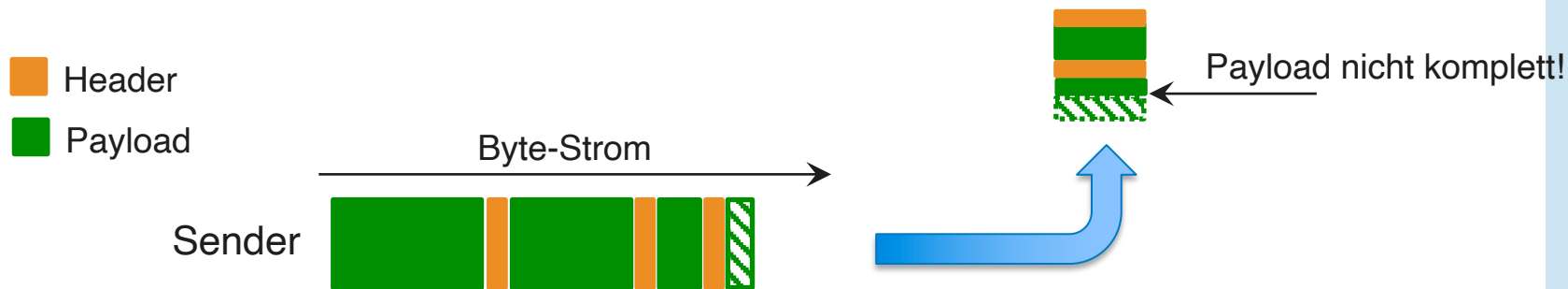
    out.println( "Echo server active, enter 'bye' to exit.\r" );
    do {
        do { // ...
        } while( buffer[idx-1] != '\r' ); // read one line, byte by byte
        // ...
    } while( true );
}
```

Aufbau von Anwendungsnachrichten

- Für jede Anwendung muss ein **Protokoll** entworfen werden
→ Format der auszutauschenden Nachrichten
- Basisformat:
 - Kopf (engl. header):
 - Inhalt: Paket-Typ (bestimmt Format der Nutzdaten) + Länge der Nutzdaten, ...
 - Nutzdaten (engl. payload)
 - Die eigentlichen Daten
- Sender wandelt Java-Objekte in Nachrichten um und verschickt diese.
- Empfänger muss Nachrichten aus dem Bytestrom erkennen und wandelt diese dann wieder in Java-Objekte um
- Für das Umwandeln von Objekten in Byte-Streams und zurück gibt es in Java das Interface `Serializable` (ist aber langsam)

TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



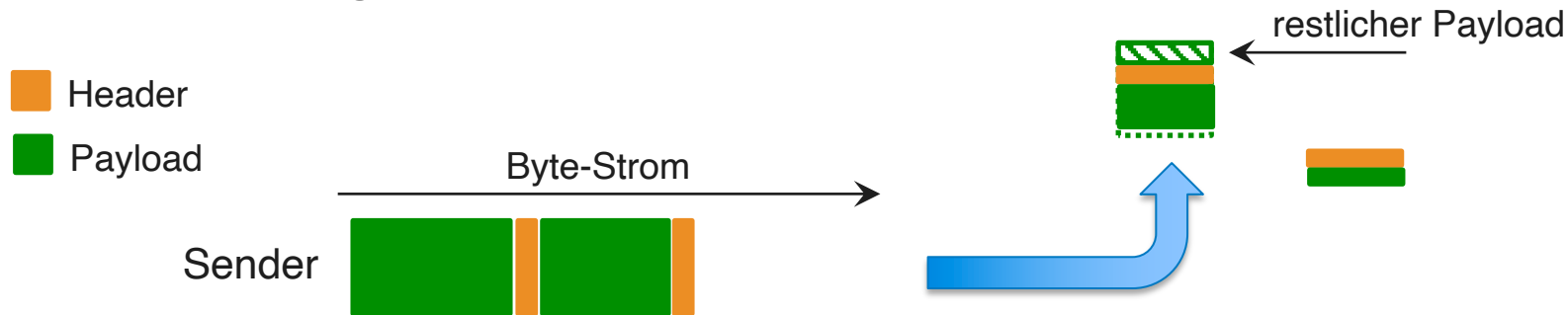
TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



TCP: Empfang von Anwendungsnachrichten

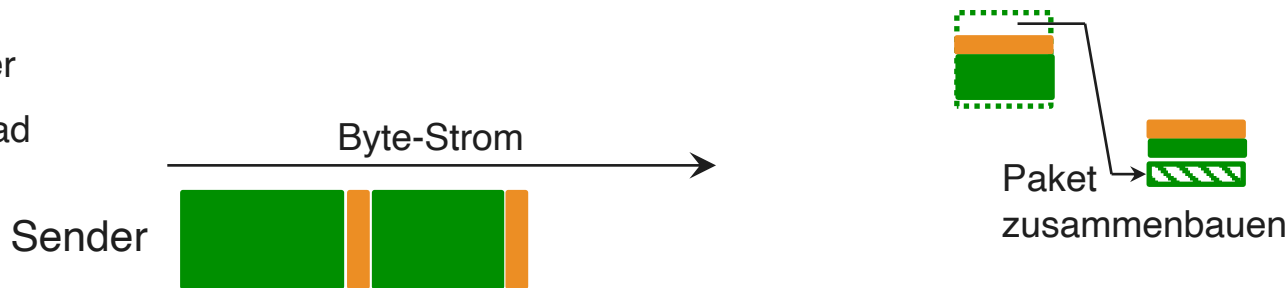
- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



TCP: Empfang von Anwendungsnachrichten

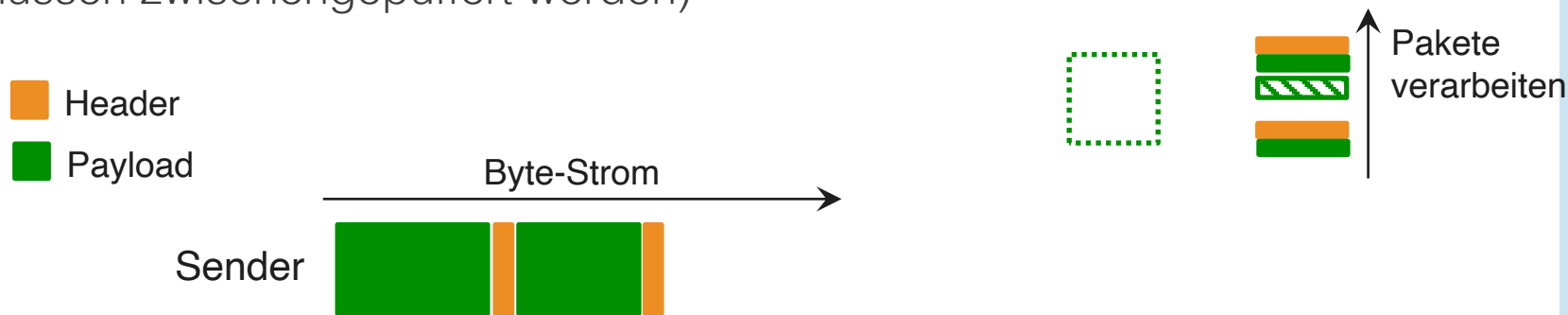
- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)

■ Header
■ Payload



TCP: Empfang von Anwendungsnachrichten

- Empfänger erhält Pakete nicht immer in einem Stück vom Socket
 - Socket verschickt einen Byte-Stream
 - IP kann diesen in Fragmente unterteilen
 - Physikalische Schicht kann weiter unterteilen (Maximum Transfer Unit Größe)
- Unter Umständen muss der Empfänger mehrere Lesezugriffe auf einen Socket verwenden, um ein Paket vollständig zu erhalten (Paketstücke müssen zwischengepuffert werden)



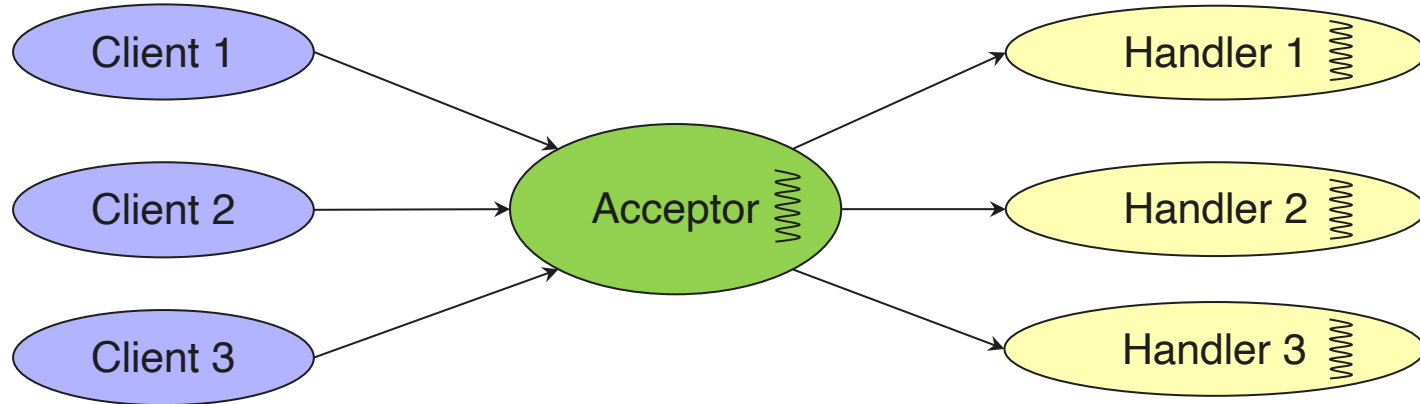
Anfrage-Antwort Zuordnung

- Bei Request-Responses (Anfrage/Antwort) wird typischerweise eine ID verwendet, um eine eindeutige Verknüpfung zu gewährleisten
 - Notwendig, falls zwischen Request/Response noch andere Nachrichten dazwischen empfangen werden
- Dieses Verhalten gibt es zum Beispiel beim X-Protokoll (X Server)
 - Der X Server schickt ereignisgetrieben zum Beispiel Maus-Ereignisse
 - Und die Anwendung schickt Zeichenbefehle zum X Server und ggf. auch Abfragen mit Antworten
 - Dies geschieht alles asynchron

4.3 Skalierbare Server-Architektur

Multi-Threaded Server für mehrere Clients (Wdlg.)

- Architektur: ein Thread pro Handler + ein Thread für den Acceptor



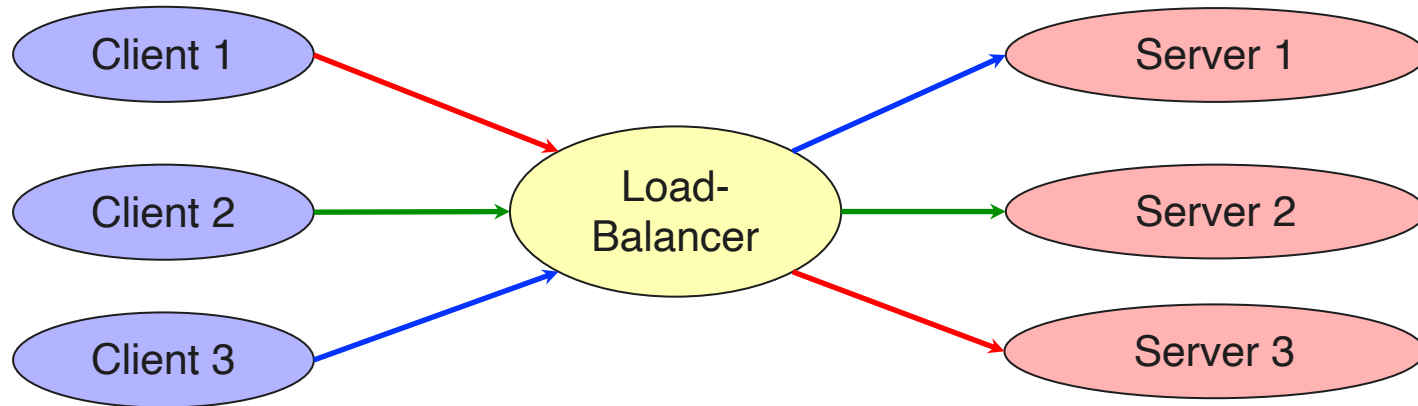
- Bei sehr vielen Clients würde solch ein Server nicht effizient arbeiten
→ **Architektur skaliert schlecht hinsichtlich der Anzahl an Clients**

Skalierbarkeit bzgl. Anzahl Clients

- Probleme:
 - Ein Thread in einer 64-Bit JVM belegt ca. 1 MB (auch wenn er inaktiv ist)
 - Erzeugen und Beenden von Threads kostet auch Zeit
 - Viele Threads belasten auch den Scheduler
 - Aufwecken blockierter Threads dauert >10 Millisekunden
- Die einfache Lösung ein Thread pro Client funktioniert ab ca. 5.000 – 10.000 nicht mehr gut (abhängig von der verwendeten Hardware)

Lösung 1: Load-Balancer

- Einsatz mehrerer Server, aber alle Clients bekommen durch einen Load-Balancer einen Server zugeteilt, an den sie ihre Anfragen schicken
- Damit kann die Anzahl an Clients pro Server reduziert werden



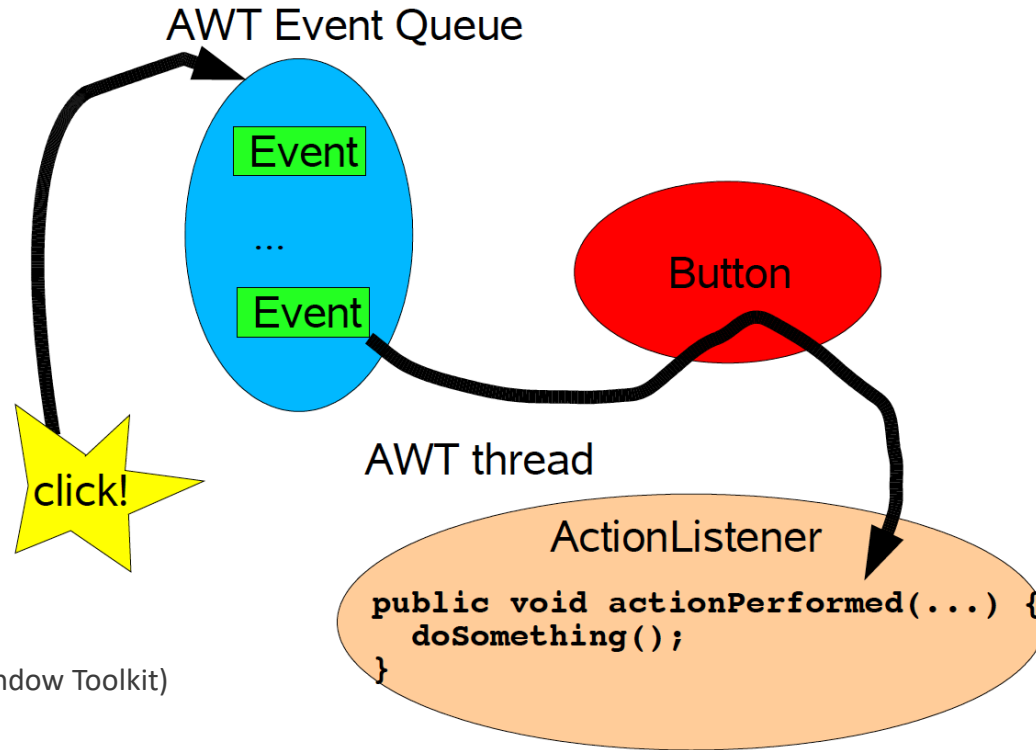
Lösung 1: Load-Balancer

- Der Ansatz wird sehr oft in der Praxis verwendet
- Der Load-Balancer wird i.d.R. nur für den ersten Zugriff verwendet, um einen Server zugeteilt zu bekommen.
- Funktioniert so einfach nur, wenn die Server replizierte Zustände haben und so jeder Server jede Anfrage beantworten kann.
- Durch die replizierten Server hat man Fehlertoleranz, aber wenn sich auf einem Server etwas ändert müssen auch die anderen Server aktualisiert werden (siehe später Kapitel „Replikation und Konsistenz“).

Lösung 2: asynchrone Sockets

- **Asynchrone Sockets** arbeiten **ereignisbasiert** (in Java NIO implementiert)
- Pro Client nach wie vor ein Socket, aber nicht ein Thread pro Socket
- Für den Datenempfang wird **kein** blockierendes `read` aufgerufen
- Stattdessen findet eine Ereignis-basierte Verarbeitung statt
- Ein Thread pollt blockierend in einer Schleife nach neuen Ereignissen (von ein oder mehrere Sockets)
- Mit jedem Ereignis ist eine Callback-Methode verknüpft

Verwandte Idee bei der GUI-Programmierung



(AWT = Abstract Window Toolkit)

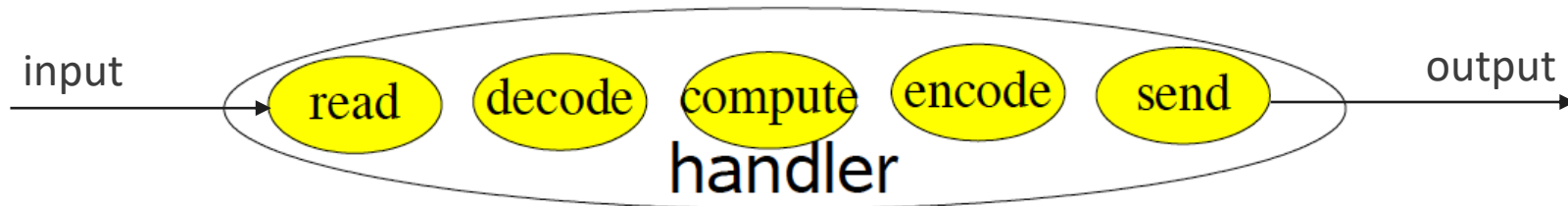
Doug Lea, State University of New York at Oswego

Lösung 2: asynchrone Sockets

- Ergebnis: 1 Thread verarbeitet die Daten von mehreren asynchronen Sockets
 - Dieser Thread holt alle Ereignisse ab, für u.U. mehrere Sockets, und ruft dann die für jedes Ereignis die registrierte Callback-Methode auf
 - Damit nicht mehr ein „teurer“ Thread pro Verbindung/Client
- Aber die Verarbeitung der Nachrichten auf dem Server darf nicht blockieren, ansonsten könnte die Abarbeitung einer Nachricht auf einer Verbindung eine andere Verbindung negativ beeinflussen

Ereignisgetriebene Server-Architektur

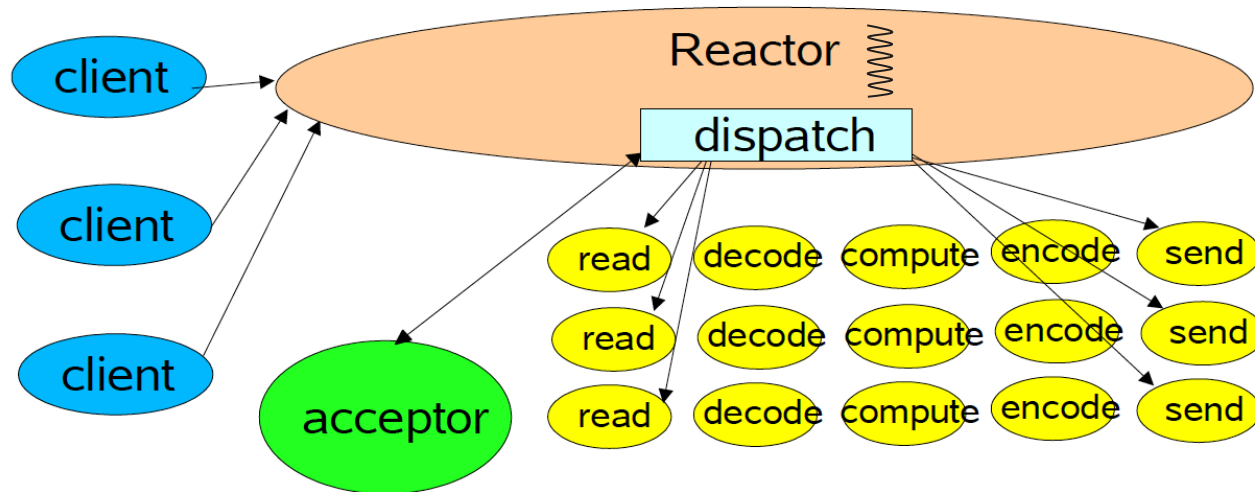
- Idee: unterteile Server-Verarbeitung (engl. handler) in nicht-blockierende Aufgaben (engl. tasks)
- Eine Task wird ausgeführt, wenn sie über ein Ereignis angestoßen wird
 - Kette der Tasks in einem Handler wird initial durch ein Empfangereignis angestoßen
 - Ablauf der Task-Kette (ähnlich Zustandsautomat) muss eingehalten werden
 - Programmierer muss Zustand des Handlers verwalten, um asynchrone Abarbeitung zu steuern



Doug Lea, State University of New York at Oswego

Stand bisher

- Server mit einem Thread, welche u.U. viele asynchrone Sockets bedient
- Ereignisgetriebene, zustandsabhängige Verarbeitung
- Handler oder Acceptor werden gerufen, wenn ein Ereignis eintritt, für dass sie sich registriert haben



Bewertung

- Bisheriger Ansatz mit einem Thread nutzt nicht Multicore-Hardware
→ von zu vielen Threads, sind wir zu nur einem Thread gewechselt
- Wir möchten gerne **mehrere Worker-Threads** die mehrere Handler parallel ausführen können
 - Aber nur beschränkt viele Threads!
 - Typischerweise insgesamt so viele Threads wie Cores vorhanden sind

Worker-Threads

- Ziel: Worker-Threads sollen den Reactor-Thread entlasten
 - Wir möchten aber nicht für jede Nachricht einen Thread erzeugen.
- Lösung: Thread-Pool (verwaltet mehrere Threads)
 - Funktionsprinzip eines Thread-Pools ähnelt einem Restaurant.
 - Anzahl der Bedienungen, die Essen zu den Gästen bringen, ist fest.
 - Produziert die Küche mehr Essen als die Bedienungen ausliefern können, stauen sich die Teller temporär. Es werden aber kurzfristig keine neuen Bedienungen eingestellt.
 - Ebenfalls werden keine entlassen, falls weniger Essen bestellt werden.

(Quelle: https://www.w3l.de/de/fileadmin/user_upload/Datenstrukturen_und_Grundlagen_der_nebenlaeufigen_Programmierung_in_Java_2014.pdf)

Beispiel: ThreadPoolExecutor

- Verwaltet intern einen Thread-Pool und eine Task-Queue
- Für die auszuführenden Tasks (`Executor`-Objekte) wird nur eine bestimmte Anzahl von Threads erzeugt.
- Stehen mehr Tasks zur Bearbeitung an, so werden diese nacheinander ab-gearbeitet.
- Hat ein Thread eine Task abgearbeitet, wird der Thread verwendet, um die die nächste Task auszuführen.

1. Beispiel: ExecutorService - Hauptklasse

- 10 Tasks werden von Thread-Pool (fixed, 5 Threads) abgearbeitet

```
public class SimpleThreadPool {  
  
    public static void main(String[] args) {  
  
        // create thread pool with fixed number of threads  
        ExecutorService executor = Executors.newFixedThreadPool(5);  
  
        // submit tasks for execution in future  
        for (int i = 0; i < 10; i++) {  
            Runnable worker = new Worker( i );  
            executor.execute( worker );  
        }  
        System.out.println("Finished all tasks.");  
  
        // Initiate shutdown -> do no longer accept new tasks  
        executor.shutdown();  
  
        // busy waiting for termination of all tasks  
        while (!executor.isTerminated()) {  
        }  
        System.out.println("Finished all tasks.");  
    }  
}
```

Sobald alle 5 Threads belegt sind,
müssen die restlichen Tasks warten.

1. Beispiel: ExecutorService - Worker

- 10 Tasks werden von Thread-Pool (fixed, 5 Threads) abgearbeitet

```
public class Worker implements Runnable {  
    private String task;  
  
    public Worker(int id){  
        this.task = "  Worker = " + id;  
    }  
  
    @Override  
    public void run() {  
        System.out.println(task + " running, using " + Thread.currentThread().getName());  
  
        try {  
            Thread.sleep(5000);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
  
        System.out.println(task + " finished, releasing " + Thread.currentThread().getName());  
    }  
}
```

Arbeit simulieren durch 5s Warten

1. Beispiel: ExecutorService - Ausgabe

```
Michaels-MacBook-Pro-Neu:pool_simple mschoett1$ java SimpleThreadPool
```

```
Executing 10 tasks with a fixed Thread-Pool with 5 threads.
```

```
Worker = 1 running, using pool-1-thread-2
```

```
Worker = 0 running, using pool-1-thread-1
```

```
Worker = 2 running, using pool-1-thread-3
```

```
Worker = 3 running, using pool-1-thread-4
```

```
Worker = 4 running, using pool-1-thread-5
```

5 Worker laufen (warten je 5s)

```
All tasks submitted.
```

10 Worker submitted

```
Worker = 1 finished, releasing pool-1-thread-2
```

```
Worker = 4 finished, releasing pool-1-thread-5
```

```
Worker = 2 finished, releasing pool-1-thread-3
```

```
Worker = 0 finished, releasing pool-1-thread-1
```

```
Worker = 3 finished, releasing pool-1-thread-4
```

5 Worker finished

```
Worker = 8 running, using pool-1-thread-1
```

```
Worker = 7 running, using pool-1-thread-3
```

```
Worker = 6 running, using pool-1-thread-5
```

```
Worker = 5 running, using pool-1-thread-2
```

```
Worker = 9 running, using pool-1-thread-4
```

Next 5 Worker running

```
Worker = 7 finished, releasing pool-1-thread-3
```

```
Worker = 9 finished, releasing pool-1-thread-4
```

```
Worker = 5 finished, releasing pool-1-thread-2
```

```
Worker = 6 finished, releasing pool-1-thread-5
```

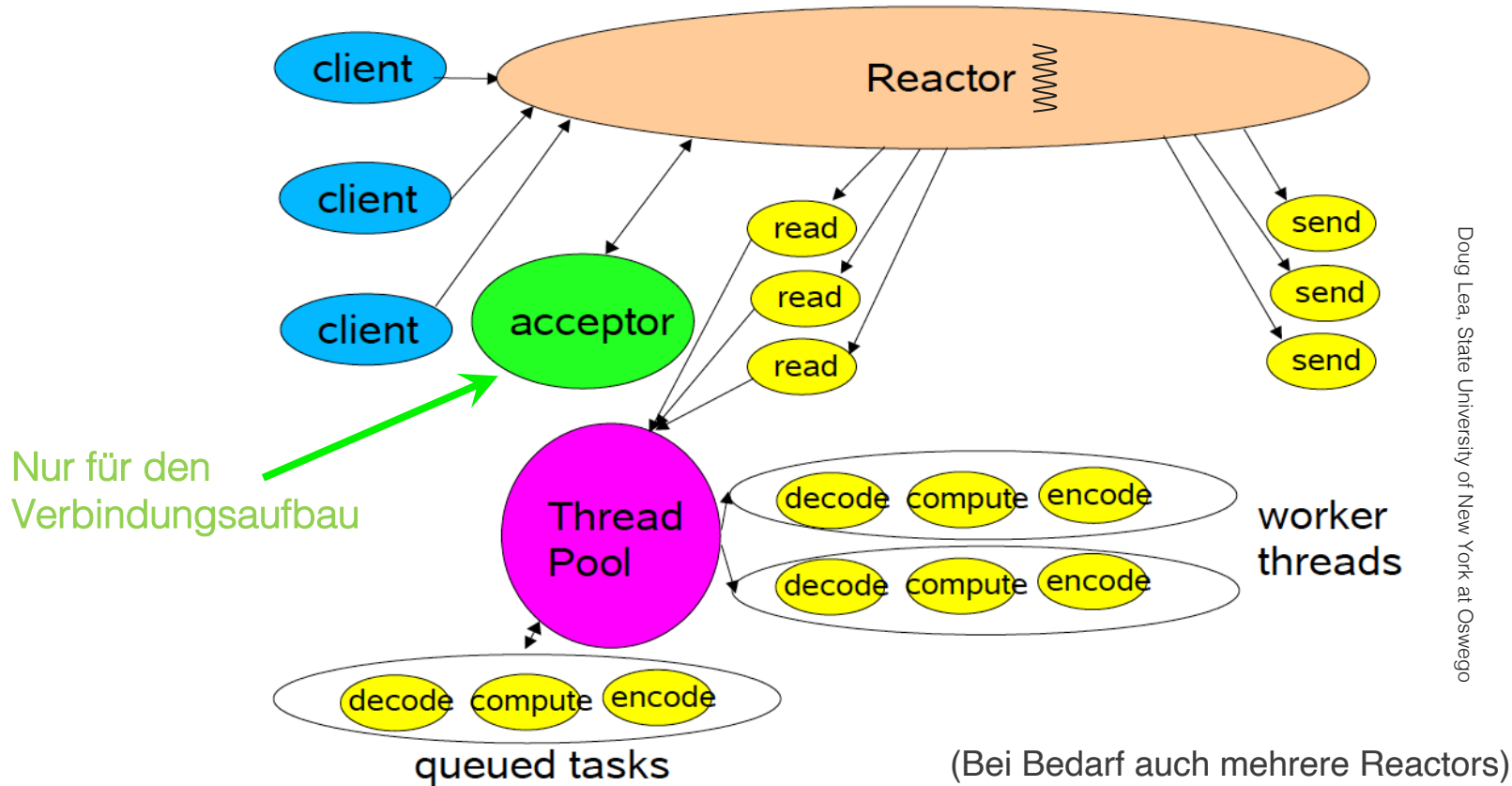
```
Worker = 8 finished, releasing pool-1-thread-1
```

Next 5 Worker finished

```
Finished all tasks.
```

```
Michaels-MacBook-Pro-Neu:pool_simple mschoett1$
```

Multi-Threaded Architektur



Doug Lea, State University of New York at Oswego

Bemerkungen zur ereignisgetriebenen Server-Architektur

- Asynchrone Programmierung erfordert Umdenken gegenüber blockierenden Threads → anspruchsvoll
- Ereignisgetriebene Logik (ähnlich einem Zustandsautomat) muss nicht-blockieren realisiert werden
→ Sperren vermeiden

Virtual Threads (Projekt Loom)

- Virtual Threads sind leichtgewichtige Threads:
 - von der JVM verwaltet und dem Betriebssystem nicht bekannt
- Erlauben es blockierende Sockets zu verwenden ohne die Skalierbarkeitsprobleme klassischer Threads
- Blockierende Socket-Operationen werden intern anders behandelt
 - JVM parkt den Virtual Thread, statt einen Thread zu blockieren
 - Intern wird also ein asynchroner Socket genutzt, dies ist aber transparent
- Funktioniert auch mit 1 Mio. Virtual Threads
- Weitere Informationen: <https://openjdk.org/projects/loom/>

4.4 Bibliotheken

Netty

- Eine über viele Jahre optimierte Bibliothek (Open-Source) für socket-basierte Kommunikation für Java-Anwendungen
- Verwendet intern asynchrone Java NIO-Sockets & Thread-Pools
- Basiert auf einer ereignis-getriebenen Architektur
 - Hohen Durchsatz (durch automatische Aggregation von Netzwerknachrichten)
 - Sowie niedrige Latenz
- Siehe: <https://netty.io/index.html>





- Open-Source C-Bibliothek¹ für sehr schnelle Netzwerke in Rechenzentren
 - Unter anderem InfiniBand (400 Gbit/s = 50 Gbyte/s & < 0.5 µs Latenz)
- Basiert auf einer ereignis-getriebenen Architektur
- Abstrahiert die Low-Level InfiniBand Verbs API mittels der UCP API²
 - Auch asynchrone Programmierung, aber keine Sockets
- Identifiziert und Verwendet (transparent) die bestmögliche Kombination an verfügbarer Hardware (z.B. InfiniBand + Shared Memory)

¹ <https://openucx.org>

² <https://openucx.readthedocs.io/en/master/api.html>

4.5 Zusammenfassung

Zusammenfassung

- Sockets sind eine Low-Level-Schnittstelle für die Kommunikation
 - erfordern explizite Behandlung von Protokollen, Pufferung und Nebenläufigkeit“
- Richtig verwendet erlauben sie aber einen schnellen Nachrichtenaustausch
 - Die Anwendung weiß, wann welche Daten wo benötigt werden
- Zudem sind sie fast für alle Plattformen und Programmiersprachen verfügbar
- Ein Thread pro blockierendem Socket funktioniert nur gut bis ca. 5.000 – 10.000 Clients, darüber sollte man auf asynchrone Sockets und Thread-Pools umstellen
- Alternativ ermöglichen Virtual Threads skalierbare Java-Server mit blockierenden Sockets